

WEB TECHNOLOGIES

EXAM STUDY MATERIAL

10-Mark Programs • 2-Mark Short Answers

DOM Event Handling • Node.js + MongoDB CRUD • AngularJS • ReactJS

NPM • REST API • SPA • Directives • Filters • Modules & Controllers • Node.js fs

Clean and organised notes for quick revision

CONTENTS

PART A — 10-MARK QUESTIONS

1	DOM Event Handling Using JavaScript	4
2	Node.js CRUD Operation Using MongoDB	8
3	Form Validation Using AngularJS	14
4	ReactJS Application	21
	The 4 Programs You Should Memorize	31
	How to Prepare	34

PART B — 2-MARK QUESTIONS

1	Installation of NPM — Steps	37
2	Role of REST API	39
3	Traditional Website vs Single Page Application	41
4	AngularJS Directives	43
5	Hello World Using ReactJS	45
6	Two-Way Data Binding in AngularJS	47
7	Filters in AngularJS	48
8	Modules and Controllers in AngularJS	50
9	File Operations in Node.js	52
	Last-Minute Memory Sheet	54

PART A — 10-MARK QUESTIONS

YOUR 4 MASTER 10-MARK QUESTIONS

No.	Topic	Master program
1	DOM Event Handling	Interactive Product/Student page
2	Node.js CRUD + MongoDB	Online Bookstore
3	AngularJS Form Validation	Retail Product Form
4	ReactJS Application	Recipe Management

These four cover almost everything your staff has identified.

1. DOM EVENT HANDLING USING JAVASCRIPT

What you need to know

For a 10-mark answer, remember these concepts:

- DOM = Document Object Model
- HTML elements become DOM objects
- JavaScript can access and modify them
- Events = user/browser actions
- Event handling = responding to events
- `getElementById()`
- `addEventListener()`
- `.value`
- `.innerHTML`
- `.style`

The **master pattern** is:

```
HTML element
  ↓
getElementById()
  ↓
addEventListener()
  ↓
User performs event
  ↓
JavaScript function executes
  ↓
DOM is modified
```

★ MASTER DOM PROGRAM

This one demonstrates:

- button click
- input
- DOM access
- changing text
- changing CSS
- calculation
- multiple event handlers

```
</> HTML
<!DOCTYPE html>
<html>
<head>
  <title>DOM Event Handling</title>

  <style>
```

```

    body {
        font-family: Arial;
        padding: 30px;
    }

    .box {
        width: 400px;
        padding: 20px;
        border: 2px solid black;
    }

    button {
        padding: 10px;
        margin: 5px;
    }
</style>
</head>

<body>

<div class="box">

    <h2 id="title">Product Calculator</h2>

    <label>Product Name:</label>
    <input type="text" id="name">

    <br><br>

    <label>Price:</label>
    <input type="number" id="price">

    <br><br>

    <label>Quantity:</label>
    <input type="number" id="quantity">

    <br><br>

    <button id="calculate">Calculate</button>
    <button id="change">Change Style</button>

    <h3 id="result"></h3>

</div>

<script>

document.getElementById("calculate").addEventListener("click", function() {

    let name = document.getElementById("name").value;

    let price = document.getElementById("price").value;

    let quantity = document.getElementById("quantity").value;

    let total = price * quantity;

```

```

        document.getElementById("result").innerHTML =
            "Product: " + name + "<br>Total Price: ₹" + total;

    });

    document.getElementById("change").addEventListener("click", function() {

        document.getElementById("title").style.color = "blue";

        document.getElementById("result").style.fontSize = "20px";

    });

</script>

</body>
</html>

```

How to adapt this to ANY question

Suppose the question says:

Develop a student result application using JavaScript DOM event handling.

Don't panic.

Change:

```

Product Name → Student Name
Price → Mark
Quantity → Subject
Total → Average/Total Mark

```

The **event-handling structure remains exactly the same**.

If they ask:

Create a ticket booking system.

Change:

```

Product → Movie
Price → Ticket Price
Quantity → Number of Tickets
Total → Ticket Cost

```

Same structure.

What to write for theory

DOM

The Document Object Model (DOM) represents an HTML document as a tree of objects. JavaScript can access and manipulate these objects to dynamically change HTML content, attributes and CSS styles.

Event

An event is an action that occurs in a web page, such as a mouse click, keyboard input or form submission.

Event handling

Event handling is the process of detecting an event and executing a function in response to that event.

`addEventListener()`

```
</> JavaScript
element.addEventListener("click", function() {
    // action
});
```

It attaches an event handler to an HTML element.

2. NODE.JS CRUD OPERATION USING MONGODB

This is probably your **highest-priority question**.

Your previous paper literally had:

```
Node.js application + MongoDB + CRUD + online bookstore.
```

So learn this one very well.

First understand CRUD

CRUD means:

```
C → Create  
R → Read  
U → Update  
D → Delete
```

For a bookstore:

```
CREATE → Add book  
READ   → Display books  
UPDATE → Change book  
DELETE → Remove book
```

Basic architecture

Understand this:

```
Browser  
  ↓  
Node.js Application  
  ↓  
MongoDB  
  ↓  
Database  
  ↓  
books collection
```

Required package

For the MongoDB Node.js driver:

```
</> Bash  
npm install mongodb
```

Your previous paper specifically asks this type of command.

★ MASTER NODE + MONGODB CRUD PROGRAM

`</>` JavaScript

```
const { MongoClient } = require("mongodb");

const url = "mongodb://127.0.0.1:27017";

const client = new MongoClient(url);

async function main() {

  try {

    // CONNECT
    await client.connect();

    console.log("MongoDB Connected");

    const db = client.db("bookstore");

    const books = db.collection("books");

    // =====
    // CREATE
    // =====

    await books.insertOne({
      title: "JavaScript Basics",
      author: "John",
      genre: "Programming",
      stock: 10
    });

    console.log("Book inserted");

    // =====
    // READ
    // =====

    const allBooks = await books.find().toArray();

    console.log("Books:");

    console.log(allBooks);

    // =====
    // UPDATE
    // =====

    await books.updateOne(
      { title: "JavaScript Basics" },
      { $set: { stock: 15 } }
    );

    console.log("Book updated");

    // =====
```

```

// DELETE
// =====

await books.deleteOne({
  title: "JavaScript Basics"
});

console.log("Book deleted");

} catch (error) {

  console.log(error);

} finally {

  await client.close();

  console.log("MongoDB connection closed");

}
}

main();

```

Understand this code instead of memorizing blindly

Connection

```

</> JavaScript

const { MongoClient } = require("mongodb");

const url = "mongodb://127.0.0.1:27017";

const client = new MongoClient(url);

await client.connect();

```

This connects Node.js to MongoDB.

Database

```

</> JavaScript

const db = client.db("bookstore");

```

Creates/accesses:

```
bookstore
```

database.

Collection

```
</> JavaScript  
  
const books = db.collection("books");
```

The collection is:

```
books
```

CREATE

```
</> JavaScript  
  
await books.insertOne({  
  title: "JavaScript Basics",  
  author: "John",  
  genre: "Programming",  
  stock: 10  
});
```

Adds a document.

READ

```
</> JavaScript  
  
const allBooks = await books.find().toArray();  
  
console.log(allBooks);
```

Retrieves documents.

UPDATE

```
</> JavaScript  
  
await books.updateOne(  
  { title: "JavaScript Basics" },  
  { $set: { stock: 15 } }  
);
```

Finds the book and changes:

```
stock: 10
```

to:

```
stock: 15
```

DELETE

```
</> JavaScript  
  
await books.deleteOne({  
  title: "JavaScript Basics"  
});
```

Deletes the matching document.

How to adapt this to different questions

Previous paper:

Online bookstore

You use:

```
books  
title  
author  
genre  
stock
```

If question becomes:

Employee management system

Change to:

```
employees  
name  
department  
salary
```

If question becomes:

Product management

Change to:

```
products  
name  
price  
category  
stock
```

If question becomes:

Student management

Change to:

```
students
name
department
mark
```

The CRUD structure **doesn't change**.

What to write in the 10-mark answer

Use this structure:

1. Definition

CRUD represents Create, Read, Update and Delete operations used to manage data in a database.

2. Connection architecture

```
Node.js → MongoDB → Database → Collection
```

3. Installation

```
</> Bash
npm install mongodb
```

4. Connection code

5. Create

6. Read

7. Update

8. Delete

9. Output

That is a strong 10-mark answer.

3. FORM VALIDATION USING ANGULARJS

This is another **staff-highlighted 10-mark question**.

The examiner can easily turn it into:

Develop an AngularJS application for validating a registration form.

or:

Develop an AngularJS application for validating employee/student/customer details.

The actual application doesn't matter much.

The important part is:

```
AngularJS
  ↓
Form
  ↓
ng-model
  ↓
Validation
  ↓
$valid / $invalid
  ↓
Error messages
```

AngularJS validation concepts you absolutely need

ng-app

Starts AngularJS.

ng-controller

Connects the controller.

ng-model

Connects input to data.

required

Makes field mandatory.

ng-minlength

Minimum characters.

ng-maxlength

Maximum characters.

\$valid

Form/input is valid.

`$invalid`

Form/input is invalid.

`$touched`

User has interacted with the field.

★ MASTER ANGULARJS FORM VALIDATION PROGRAM

`</>` HTML

```
<!DOCTYPE html>
<html>

<head>

  <title>AngularJS Form Validation</title>

  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js">
  </script>

  <style>

    body {
      font-family: Arial;
      padding: 30px;
    }

    .box {
      width: 400px;
      padding: 20px;
      border: 2px solid black;
    }

    input {
      width: 90%;
      padding: 8px;
      margin: 5px;
    }

    .error {
      color: red;
    }

    button {
      padding: 10px 20px;
    }

  </style>

</head>

<body ng-app="myApp">

<div ng-controller="formController">

  <div class="box">

    <h2>Registration Form</h2>
```

```

<form name="myForm" ng-submit="submitForm()" novalidate>

  <!-- NAME -->

  <label>Name:</label>

  <input
    type="text"
    name="name"
    ng-model="user.name"
    required
    ng-minlength="3">

  <p class="error"
    ng-show="myForm.name.$touched && myForm.name.$invalid">

    Name must contain at least 3 characters.

  </p>

  <!-- EMAIL -->

  <label>Email:</label>

  <input
    type="email"
    name="email"
    ng-model="user.email"
    required>

  <p class="error"
    ng-show="myForm.email.$touched && myForm.email.$invalid">

    Enter a valid email address.

  </p>

  <!-- AGE -->

  <label>Age:</label>

  <input
    type="number"
    name="age"
    ng-model="user.age"
    required
    min="18"
    max="60">

  <p class="error"
    ng-show="myForm.age.$touched && myForm.age.$invalid">

    Age must be between 18 and 60.

  </p>

  <!-- PASSWORD -->

```

```

<label>Password:</label>

<input
  type="password"
  name="password"
  ng-model="user.password"
  required
  ng-minlength="6">

<p class="error"
  ng-show="myForm.password.$touched &&
    myForm.password.$invalid">

    Password must contain at least 6 characters.

</p>

<br>

<button type="submit"
  ng-disabled="myForm.$invalid">

  Submit

</button>

</form>

<h3>{{message}}</h3>

</div>

</div>

<script>

var app = angular.module("myApp", []);

app.controller("formController", function($scope) {

  $scope.user = {};

  $scope.message = "";

  $scope.submitForm = function() {

    if ($scope.myForm.$valid) {

      $scope.message =
        "Form submitted successfully!";

    }

  };

});

```

```
</script>

</body>

</html>
```

Understand the important pieces

AngularJS starts here

```
</> HTML

<body ng-app="myApp">
```

Controller

```
</> HTML

<div ng-controller="formController">
```

Form

```
</> HTML

<form name="myForm">
```

AngularJS gives the form an object:

```
myForm
```

Input binding

```
</> HTML

<input
  ng-model="user.name"
  required>
```

`ng-model` connects the input with:

```
user.name
```

Validation

This:

```
</> HTML

required
```

means:

The field cannot be empty.

This:

```
</> HTML
ng-minlength="3"
```

means:

Minimum 3 characters.

This:

```
</> HTML
type="email"
```

makes AngularJS validate the input as an email.

\$invalid

```
</> JavaScript
myForm.name.$invalid
```

means the name field is currently invalid.

\$valid

```
</> JavaScript
myForm.$valid
```

means the entire form is valid.

\$touched

```
</> JavaScript
myForm.name.$touched
```

means the user has interacted with the field.

So:

```
</> HTML
ng-show="myForm.name.$touched && myForm.name.$invalid"
```

means:

Show the error only after the user has interacted with the field and the value is invalid.

How to adapt this to any question

Suppose question:

Develop an AngularJS application to validate a student registration form.

Change:

```
Name  
Email  
Age  
Password
```

to whatever fields the question asks.

For:

Employee registration

Use:

```
Employee Name  
Email  
Salary  
Department
```

For:

Product form

Use:

```
Product Name  
Price  
Quantity  
Category
```

The validation mechanism remains the same.

4. REACTJS APPLICATION

This is the other huge question.

Your staff specifically said:

ReactJS application
JSX, props, state, event handling, CSS styling

And your previous paper asked for a **recipe management application** with:

- View recipes
- Add recipes
- Edit recipes
- Delete recipes
- Filter recipes

So we're going to create one master React application containing **all of these concepts**.

React concepts you need

JSX

JavaScript XML.

Allows you to write HTML-like syntax inside JavaScript.

```
</>   JavaScript (JSX)
<h1>Hello</h1>
```

Component

A reusable part of the UI.

```
</>   JavaScript (JSX)

function App() {
  return <h1>Hello</h1>;
}
```

Props

Data passed from parent component to child component.

```
Parent
  ↓ props
Child
```

Props are generally read-only.

State

Data maintained by a component.

```
</> JavaScript (JSX)

const [recipes, setRecipes] = useState([]);
```

When state changes, React re-renders the component.

Event handling

```
</> JavaScript (JSX)

<button onClick={handleClick}>
```

React uses events such as:

```
onClick
onChange
onSubmit
```

CSS styling

Can use a CSS file:

```
</> JavaScript (JSX)

import './App.css';
```

★ MASTER REACT APPLICATION

For exam purposes, this is the kind of application I recommend learning.

It contains:

- JSX
- Component
- Props
- State
- Events
- Form
- Add
- Delete
- Search/filter
- CSS

```
</> JavaScript (JSX)

import React, { useState } from "react";
import './App.css';

function Recipe({ recipe, onDelete }) {
```

```

    return (
      <div className="card">
        <h3>{recipe.title}</h3>
        <p>{recipe.description}</p>
        <p>
          Cooking Time: {recipe.time} minutes
        </p>
        <button
          onClick={() => onDelete(recipe.id)}>
          Delete
        </button>
      </div>
    );
  }
}

function App() {
  const [recipes, setRecipes] = useState([
    {
      id: 1,
      title: "Pizza",
      description: "Cheese pizza",
      time: 30
    },
    {
      id: 2,
      title: "Pasta",
      description: "Creamy pasta",
      time: 20
    }
  ]);

  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");
  const [time, setTime] = useState("");
  const [search, setSearch] = useState("");

  function addRecipe(event) {
    event.preventDefault();

    if (title === "" || description === "" || time === "") {
      return;
    }

    const newRecipe = {

```

```

        id: Date.now(),

        title: title,

        description: description,

        time: time

    };

    setRecipes([...recipes, newRecipe]);

    setTitle("");

    setDescription("");

    setTime("");
}

function deleteRecipe(id) {

    setRecipes(
        recipes.filter(recipe => recipe.id !== id)
    );

}

const filteredRecipes = recipes.filter(recipe =>
    recipe.title
        .toLowerCase()
        .includes(search.toLowerCase())
);

return (

    <div className="container">

        <h1>Recipe Management</h1>

        {/* SEARCH */}

        <input
            type="text"
            placeholder="Search recipe"
            value={search}
            onChange={(e) => setSearch(e.target.value)}
        />

        {/* ADD FORM */}

        <form onSubmit={addRecipe}>

            <input
                type="text"
                placeholder="Recipe title"
                value={title}
                onChange={(e) => setTitle(e.target.value)}
            />
        </form>
    </div>
);

```

```

    />

    <input
      type="text"
      placeholder="Description"
      value={description}
      onChange={(e) =>
        setDescription(e.target.value)
      }
    />

    <input
      type="number"
      placeholder="Cooking time"
      value={time}
      onChange={(e) =>
        setTime(e.target.value)
      }
    />

    <button type="submit">
      Add Recipe
    </button>

  </form>

  { /* RECIPE LIST */ }

  <div>

    {filteredRecipes.map(recipe => (

      <Recipe
        key={recipe.id}
        recipe={recipe}
        onDelete={deleteRecipe}
      />

    ))}

  </div>

</div>

);
}

export default App;

```

CSS for the React application

Create:

App.css

`</>` CSS

```
body {
  font-family: Arial;
  background: #f2f2f2;
}

.container {
  width: 700px;
  margin: 30px auto;
  padding: 20px;
  background: white;
}

input {
  padding: 10px;
  margin: 5px;
}

button {
  padding: 10px 15px;
  margin: 5px;
  cursor: pointer;
}

.card {
  border: 1px solid #ccc;
  padding: 15px;
  margin-top: 15px;
}
```

Now understand how this single React program covers your syllabus

This is important.

Don't just memorize the code.

Know **where** each exam keyword appears.

JSX

This entire:

`</>` JavaScript (JSX)

```
return (
  <div className="container">
    <h1>Recipe Management</h1>
  </div>
);
```

is JSX.

Props ★★★★★

Look at:

```
</> JavaScript (JSX)

function Recipe({ recipe, onDelete }) {
```

The parent sends:

```
</> JavaScript (JSX)

<Recipe
  recipe={recipe}
  onDelete={deleteRecipe}
/>
```

These are **props**.

The flow is:

```
App
↓
Recipe component
↓
props
↓
recipe
onDelete
```

State ★★★★★

This:

```
</> JavaScript (JSX)

const [recipes, setRecipes] = useState([]);
```

is state.

Other states:

```
</> JavaScript (JSX)

const [title, setTitle] = useState("");

const [description, setDescription] = useState("");

const [time, setTime] = useState("");

const [search, setSearch] = useState("");
```

When state changes, React updates the UI.

Event handling ★★★★★

Here:

```
</> JavaScript (JSX)
onClick={() => onDelete(recipe.id)}
```

handles a click.

Here:

```
</> JavaScript (JSX)
onChange={(e) => setTitle(e.target.value)}
```

handles input changes.

Here:

```
</> JavaScript (JSX)
onSubmit={addRecipe}
```

handles form submission.

So you've covered React event handling.

CSS styling

We import:

```
</> JavaScript (JSX)
import './App.css';
```

and use:

```
</> JavaScript (JSX)
className="container"
```

```
</> JavaScript (JSX)
className="card"
```

The CSS file styles them.

Forms

This:

```
</> JavaScript (JSX)
<form onSubmit={addRecipe}>
```

creates the form.

And:

```
</> JavaScript (JSX)
event.preventDefault();
```

prevents the browser from reloading the page when the form is submitted.

Add operation

```
</> JavaScript (JSX)
setRecipes([...recipes, newRecipe]);
```

adds the new recipe to the existing state.

Delete operation

```
</> JavaScript (JSX)
setRecipes(
  recipes.filter(recipe => recipe.id !== id)
);
```

removes the selected recipe.

Search/filter

```
</> JavaScript (JSX)
const filteredRecipes = recipes.filter(recipe =>
  recipe.title
    .toLowerCase()
    .includes(search.toLowerCase())
);
```

This filters recipes according to the search text.

★ If the question changes

Suppose the exam asks:

Build a ReactJS application for managing employees.

Don't try to create a new program from scratch.

Take the recipe program and change:

```
Recipe
  ↓
Employee

title
  ↓
name

description
  ↓
department

time
  ↓
salary
```

The React concepts remain exactly the same.

VERY IMPORTANT: React setup

If the question asks for the **steps to create a React application**, your syllabus/old paper may expect the traditional setup:

```
</>  Bash

npx create-react-app myapp
cd myapp
npm start
```

Then:

```
src/
├── App.js
├── App.css
└── index.js
```

Depending on what your staff teaches, they may expect `create-react-app` because your previous paper specifically asks about the React toolchain.

THE 4 PROGRAMS YOU SHOULD ACTUALLY MEMORIZE

Don't memorize every line separately.

Memorize these four **structures**.

PROGRAM 1 — DOM

```
HTML
↓
getElementById()
↓
addEventListener()
↓
click/input event
↓
function
↓
innerHTML / style / value
```

Master:

```
</> JavaScript

element.addEventListener("click", function() {
  element.innerHTML = "...";
});
```

PROGRAM 2 — NODE + MONGODB

Memorize this structure:

```
require mongodb
↓
MongoClient
↓
connect()
↓
db()
↓
collection()
↓
insertOne()
find()
updateOne()
deleteOne()
↓
close()
```

The four lines you absolutely cannot forget:

```
</> JavaScript
```

```
insertOne()  
find()  
updateOne()  
deleteOne()
```

That's CRUD.

PROGRAM 3 — ANGULARJS

Memorize this structure:

```
ng-app  
  ↓  
module  
  ↓  
controller  
  ↓  
$scope  
  ↓  
ng-model  
  ↓  
form validation  
  ↓  
$valid / $invalid
```

Important validation attributes:

```
required  
ng-minlength  
ng-maxlength  
type="email"  
min  
max
```

PROGRAM 4 — REACT

Memorize:

```
Component  
  ↓  
JSX  
  ↓  
State  
  ↓  
Props  
  ↓  
Events  
  ↓  
Forms  
  ↓  
CSS
```

The core patterns:

State

```
</> JavaScript (JSX)  
  
const [data, setData] = useState([]);
```

Props

```
</> JavaScript (JSX)  
  
<Child data={data} />
```

Event

```
</> JavaScript (JSX)  
  
<button onClick={handleClick}>
```

Input

```
</> JavaScript (JSX)  
  
<input  
  value={name}  
  onChange={(e) => setName(e.target.value)}  
/>
```

CSS

```
</> JavaScript (JSX)  
  
import './App.css';
```

How I would prepare if I were writing your exam

You have **four 10-mark topics**.

I'd allocate your study time like this:

- **1. Node.js + MongoDB CRUD**

Highest priority

Be able to write the entire program.

- **2. ReactJS Application**

Be able to write the entire application and explain JSX, props, state, events and CSS.

- **3. AngularJS Form Validation**

Be able to write the complete form and explain `ng-model` , `$valid` , `$invalid` , etc.

- **4. DOM Event Handling**

Be able to write the complete HTML + JavaScript program.

One more important thing

You **don't need four completely different programs for four possible scenarios**.

For example, if the paper says:

"Develop a movie ticket booking application using AngularJS."

Your Angular master program already gives you:

```
ng-app
ng-controller
ng-model
form
validation
$scope
```

You simply change the fields and calculations.

If it says:

"Develop a retail product price application."

Same thing.

If React says:

"Build a recipe management application."

Use the recipe master.

If it says:

"Build an employee management application."

Use the same React architecture and change the data.

The examiner is changing the story, not the underlying programming concepts.

So your goal is not to memorize 10 applications.

Master these 4 architectures.

And when we continue, I can take these **four master programs one at a time and explain every line that is actually worth understanding for the exam**, rather than teaching the entire framework.

PART B — 2-MARK QUESTIONS

Perfect. Since these are **strictly your 2-mark topics**, we should prepare them differently from the 10-mark programs.

For each one, I'll give you:

1. What the concept means
2. What to write for 2 marks
3. Important syntax/example
4. What keywords you must remember

You don't need huge explanations for these. The goal is: **see the question → immediately know what to write.**

1. Installation of NPM — Steps ★★★★★

First: What is NPM?

NPM = Node Package Manager.

It is the package manager used with Node.js to install, manage and share JavaScript packages/modules.

Installation steps

First install **Node.js** from the official Node.js website. NPM is normally installed along with Node.js.

Verify installation:

```
</> Bash
node -v
```

Check NPM:

```
</> Bash
npm -v
```

Create a project:

```
</> Bash
mkdir myapp
cd myapp
```

Initialize the project:

```
</> Bash
npm init -y
```

Install a package:

```
</> Bash
npm install mongodb
```

★ 2-mark answer

NPM (Node Package Manager) is used to install and manage packages required by Node.js applications. After installing Node.js, NPM can be verified using ``npm -v``. A project can be initialized using ``npm init -y``, and packages can be installed using ``npm install package-name``.

Remember:

```
node -v  
npm -v  
npm init -y  
npm install package-name
```

2. Role of REST API ★★★★★

What is REST?

REST = Representational State Transfer.

A REST API allows different applications to communicate over HTTP.

For example:

```
React Application
  ↓
  REST API
  ↓
Node.js Server
  ↓
  MongoDB
```

Common HTTP methods:

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Delete data

Example

```
GET /students
```

gets students.

```
POST /students
```

creates a student.

★ 2-mark answer

A REST API enables communication between client and server using HTTP. It allows resources to be created, retrieved, updated and deleted using methods such as POST, GET, PUT and DELETE. REST APIs are commonly used to connect web applications with server-side applications and databases.

Remember:

```
GET    → Read
POST   → Create
PUT    → Update
DELETE → Delete
```

This is directly connected to your **Node.js + MongoDB CRUD** 10-mark question.

3. Traditional Website vs Single Page Application ★★★★★

This is very easy if you remember the fundamental difference.

Traditional Website

When moving between pages, the browser generally requests and loads another HTML page.

```
Page 1
↓
Reload
↓
Page 2
↓
Reload
↓
Page 3
```

SPA

SPA = Single Page Application

The application loads a main page and JavaScript dynamically updates the content without requiring a complete page reload for every action.

```
One main page
↓
JavaScript
↓
Content changes dynamically
```

Comparison

Traditional Website	SPA
Multiple page loads	Mainly one page
Full-page reloads	Dynamic updates
Navigation can involve server requests for new pages	JavaScript updates the view
Simpler traditional architecture	Rich interactive UI

★ 2-mark answer

A traditional website loads separate HTML pages and may perform a full-page reload during navigation. A Single Page Application (SPA) loads one main page and dynamically updates its content using JavaScript without completely reloading the page.

Examples of technologies associated with SPA

AngularJS
React
Vue

4. AngularJS Directives ★★★★★

This is **very important** because your previous paper directly tested directives.

What is a directive?

An AngularJS directive is a special instruction added to HTML that extends HTML's functionality.

They usually begin with:

```
ng -
```

Important directives

```
</> HTML
```

```
ng-app  
ng-controller  
ng-model  
ng-init  
ng-repeat  
ng-click  
ng-show  
ng-hide  
ng-bind
```

What they do

Directive	Purpose
ng-app	Defines AngularJS application
ng-controller	Specifies controller
ng-model	Provides data binding
ng-init	Initializes data
ng-repeat	Repeats HTML elements
ng-click	Handles click
ng-show	Shows element
ng-hide	Hides element
ng-bind	Binds/displays data

Example

```
</> HTML

<div ng-app="myApp" ng-controller="myCtrl">

  <input ng-model="name">

  <p>{{name}}</p>

</div>
```

★ 2-mark answer

AngularJS directives are special instructions added to HTML that extend its functionality and provide AngularJS features. Examples include `ng-app`, `ng-controller`, `ng-model`, `ng-repeat` and `ng-click`.

If they ask:

Which directive binds HTML controls to data?

Answer:

ng-model

Which directive initializes AngularJS?

ng-app

5. Hello World Using ReactJS ★★★★★

This is a very likely short question because your previous paper literally had it.

Basic React program

```
</> JavaScript (JSX)

import React from "react";
import ReactDOM from "react-dom/client";

function App() {
  return <h1>Hello World</h1>;
}

const root = ReactDOM.createRoot(
  document.getElementById("root")
);

root.render(<App />);
```

HTML:

```
</> HTML

<div id="root"></div>
```

Output

```
Hello World
```

Understand the code

This:

```
</> JavaScript (JSX)

function App() {
  return <h1>Hello World</h1>;
}
```

is a React component.

This:

```
</> JavaScript (JSX)

<h1>Hello World</h1>
```

is **JSX**.

This:

```
</> JavaScript
ReactDOM.createRoot(...)
```

creates the React root.

This:

```
</> JavaScript
root.render(<App />);
```

renders the component into the DOM.

★ 2-mark answer

ReactJS can display Hello World by creating a functional component that returns JSX and rendering it into the root DOM element using `ReactDOM.createRoot()` and `root.render()`.

Remember:

```
Component → JSX → createRoot() → render()
```

6. Two-Way Data Binding in AngularJS ★★★★★

This is one of your **most important 2-mark questions**.

Two-way binding means:

```
Model
  ↓
ng-model
  ↓
View
```

Data flows in **both directions**.

Example

```
</> HTML

<input ng-model="name">

<p>Hello {{name}}</p>
```

If the model changes:

```
</> JavaScript

$scope.name = "Ajay";
```

the view updates.

If the user types something into the input:

```
Input → Model
```

the model also updates.

★ **2-mark answer**

Two-way data binding in AngularJS synchronizes the model and the view. Changes in the model are automatically reflected in the view, and changes made by the user in the view update the model. The `ng-model` directive is used for two-way binding.

Most important keyword:

```
ng-model
```

Easy memory:

```
Model ↔ View
```

7. Filters in AngularJS ★★★★★

A filter is used to **format or transform data before displaying it**.

For example:

```
</> HTML
{{name | uppercase}}
```

If:

```
name = "ajay"
```

output:

```
AJAY
```

Important built-in filters

Filter	Purpose
<code>uppercase</code>	Converts to uppercase
<code>lowercase</code>	Converts to lowercase
<code>currency</code>	Formats currency
<code>number</code>	Formats numbers
<code>date</code>	Formats dates
<code>orderBy</code>	Sorts data
<code>filter</code>	Filters data
<code>limitTo</code>	Limits displayed data

Examples

```
</> HTML
{{name | uppercase}}
```

```
</> HTML
{{price | currency}}
```

```
</> HTML
{{name | lowercase}}
```

```
</> HTML
```

```
{{students | orderBy:'name'}}
```

★ 2-mark answer

Filters in AngularJS are used to format or transform data before displaying it in the view. Common built-in filters include `uppercase`, `lowercase`, `currency`, `number`, `date`, `orderBy`, `filter` and `limitTo`.

Remember:

```
uppercase  
lowercase  
currency  
number  
date  
orderBy  
filter  
limitTo
```

8. Modules and Controllers in AngularJS ★★★★★

This looks complicated but it's actually simple.

Module

A module is a **container for an AngularJS application and its components**.

Example:

```
</> JavaScript
var app = angular.module("myApp", []);
```

Controller

A controller contains the **application logic and data**.

```
</> JavaScript
app.controller("myCtrl", function($scope) {
    $scope.name = "Ajay";
});
```

HTML:

```
</> HTML
<div ng-app="myApp" ng-controller="myCtrl">
    {{name}}
</div>
```

Output:

```
Ajay
```

Relationship

Remember:

```
Module
  ↓
Controller
  ↓
$scope
  ↓
View
```

★ 2-mark answer

A module is a container that organizes an AngularJS application and its components. A controller contains the application's logic and data and communicates with the view using `\$scope`.

Important syntax

```
</> JavaScript
var app = angular.module("myApp", []);

app.controller("myCtrl", function($scope) {
    $scope.name = "Ajay";
});
```

9. File Operations in Node.js ★★★★★

Node.js provides the **File System (`fs`)** module for working with files.

First import it:

```
</> JavaScript
const fs = require("fs");
```

Create/Write a file

```
</> JavaScript
fs.writeFile("data.txt", "Hello World", function(err) {
    if (err) throw err;
    console.log("File created");
});
```

Read a file

```
</> JavaScript
fs.readFile("data.txt", "utf8", function(err, data) {
    if (err) throw err;
    console.log(data);
});
```

Append data

```
</> JavaScript
fs.appendFile("data.txt", "Welcome", function(err) {
    if (err) throw err;
    console.log("Data added");
});
```

Delete a file

```
</> JavaScript
fs.unlink("data.txt", function(err) {
    if (err) throw err;
    console.log("File deleted");
});
```

★ The four operations you MUST remember

```
writeFile()
  ↓
Create / Write

readFile()
  ↓
Read

appendFile()
  ↓
Add data

unlink()
  ↓
Delete
```

★ 2-mark answer

Node.js provides the built-in `fs` (File System) module for performing file operations. Common methods are `fs.writeFile()` for writing, `fs.readFile()` for reading, `fs.appendFile()` for adding data and `fs.unlink()` for deleting a file.

Important syntax:

```
</> JavaScript
const fs = require("fs");
```

YOUR 9 QUESTIONS — LAST-MINUTE MEMORY SHEET

If you're revising just before entering the exam, remember this:

1. NPM

```
npm = Node Package Manager

node -v
npm -v
npm init -y
npm install package
```

2. REST API

```
GET    → Read
POST   → Create
PUT    → Update
DELETE → Delete
```

3. Traditional vs SPA

```
Traditional → multiple pages + reload
SPA → one main page + dynamic update
```

4. AngularJS Directives

```
ng-app
ng-controller
ng-model
ng-init
ng-repeat
ng-click
ng-show
ng-hide
ng-bind
```

5. React Hello World

```
Component
↓
JSX
↓
createRoot()
↓
render()
```

6. Two-way binding

```
Model ↔ View
↑
ng-model
```

7. AngularJS Filters

```
uppercase  
lowercase  
currency  
number  
date  
orderBy  
filter  
limitTo
```

8. AngularJS Module + Controller

```
Module  
↓  
Controller  
↓  
$scope  
↓  
View
```

</> JavaScript

```
angular.module("myApp", []);  
  
app.controller("myCtrl", function($scope) {  
    $scope.name = "Ajay";  
});
```

9. Node.js File Operations

```
fs.writeFile()  
fs.readFile()  
fs.appendFile()  
fs.unlink()
```

What I'd memorize word-for-word

There are **five particularly likely 2-mark answers** from your previous paper/staff list:

1. Role of REST API
2. AngularJS directives
3. Two-way data binding
4. Filters in AngularJS
5. Modules and controllers

And the **five syntax-heavy ones**:

```
npm commands  
React Hello World  
ng-model  
AngularJS directives  
Node.js fs operations
```

If you understand the **9 blocks above**, you have essentially covered the exact short-question list your staff gave you.